Reusable Mathematics for Collider Research

SCIENTIFIC WHITE PAPER | IMPLEMENTED FEASIBILITY DEMONSTRATOR

Real paceval C++ integration, deterministic tests and independent numerical validation

paceval UG · Munich, Germany · 10 November 2026

Verified research outcome. Explicit mathematical collider-analysis kernels were implemented as reusable paceval computation graphs, executed through the original Linux runtime and validated against independently constructed references. Four regular graphs, two TINC-enabled graphs, five synthetic parameter variations and a three-graph batch call were exercised successfully. All **24 of 24** selected TINC intervals included the independently computed 110-decimal-digit reference value. These results demonstrate the technical feasibility of the specified use case on synthetic data.

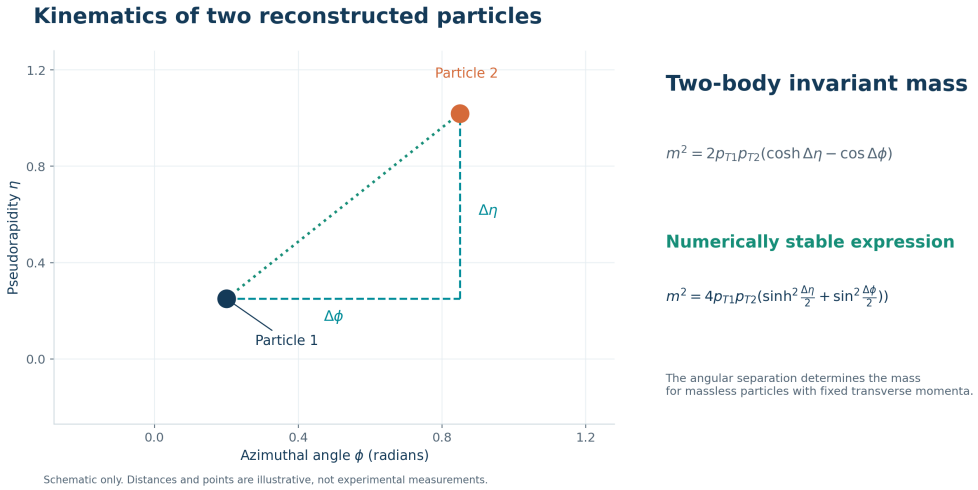


Figure 1. Collider-inspired kinematic geometry, shown schematically. Not a CERN event display.

1. Research question and scope

Modern collider analysis repeatedly evaluates defined mathematical expressions for kinematics, selections and calibration-like quantities. The question of this demonstrator is concrete: **can the documented paceval execution-graph API be used as a**

reusable classical mathematical component for selected collider-analysis kernels, and can the obtained values be independently verified?

A C++ program answers this for a small set of fully explicit formulas. It uses deterministic synthetic data, not reconstructed collision events from CERN, and the supplied event weight is illustrative rather than a detector-derived correction. The experiment does not implement track fitting, event reconstruction, detector simulation, ROOT I/O or a complete physics-analysis pipeline.

The distribution provides one current companion README, the C++ implementation, the CMake test configuration, the independent high-precision script, input and validation examples and the figures embedded here.

2. Implemented mathematical computations

Two massless particles of transverse momenta `pt1` and `pt2` (GeV) and angular differences `deta` and `dphi` have the squared invariant mass

```
m2_direct = 2*pt1*pt2*(cosh(deta)-cos(dphi))
m2_stable = 4*pt1*pt2*(sinh(deta/2)^2+sin(dphi/2)^2)
```

These formulas are algebraically identical under the massless-particle approximation. The stable representation avoids subtracting nearly equal values when the particles are almost collinear. Both formulas are implemented as **separate reusable paceval graphs**, so their behavior can be examined directly.

The additional eight-variable synthetic event-weight model is:

```
W = (1 + a*exp(-abs(deta))*cos(dphi)
       + b*ln(1+pt1/100)
       + c*sin(pt2/30)^2)
    * (1 + d*tanh((pt1+pt2-100)/60))
```

Here `a,b,c,d` are illustrative parameters, not detector calibration constants. The literal `ln` is required by the paceval expression language for the natural logarithm.

Graph handle	Actual executable expression or task	Purpose
G1	Direct mass squared	Define a cancellation-sensitive reference expression

Graph handle	Actual executable expression or task	Purpose
G2	Stable mass squared	Evaluate an equivalent formulation with improved numerical conditioning
G3	Synthetic event weight	Reuse a parameterized graph over events and scenario settings
G4	Stable mass squared × weight	Evaluate a fused explicit scalar expression
T1, T2	Both mass formulas with TINC enabled	Obtain lower and upper interval outputs for independent checking

The program initializes the actual shared runtime and constructs **four ordinary handles once per run**, plus **two separate TINC handles** for the interval study. The standard batch API evaluates each graph for batches of event inputs. A separate API call evaluates G1, G2 and G3 together. The same G3 handle is used for five parameter variations (`0.8`, `0.9`, `1.0`, `1.1`, `1.2` times each parameter), both as individual scenarios and as one flattened input sequence. All of these are executed paths in the accompanying source, not proposed pseudocode.

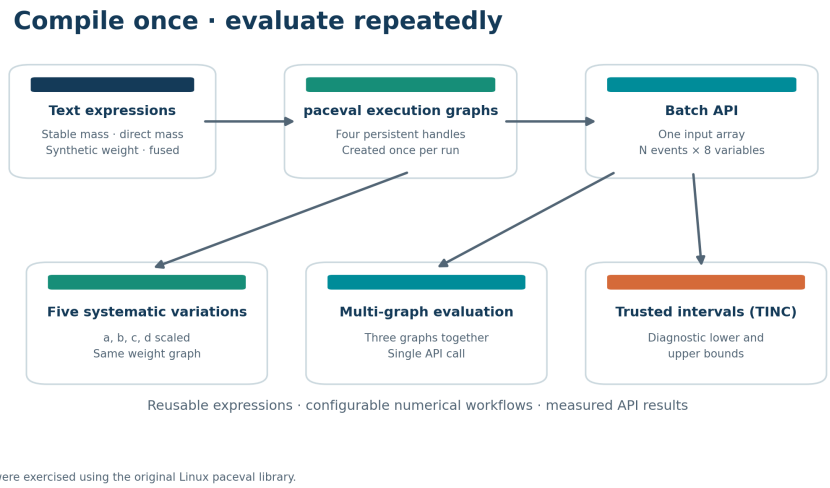


Figure 2. Four persistent mathematical graphs, joint evaluation, synthetic parameter variations and TINC verification.

3. Concrete paceval API integration

The program uses `dlopen` to load a separately supplied original paceval Linux shared library and resolves these documented symbols:

- `pacevalLibrary_Initialize` and `pacevalLibrary_Free` for lifecycle management;

- `pacevalLibrary_CreateComputation` and `pacevalLibrary_DeleteComputation` for persistent mathematical handles;
- `pacevalLibrary_dGetComputationResultExt` for event-batch evaluation;
- `pacevalLibrary_dGetMultipleComputationsResultsExt` for joint evaluation of three mathematical graphs;
- `pacevalLibrary_dGetComputationResult` for selected TINC output triples;
- `pacevalLibrary_GetIsError` and `pacevalLibrary_GetErrorInformation` for status diagnostics.

The exact expressions, variable order, memory layout and validation thresholds are documented in src/collider.cpp. No substitute expression engine is used in the real-runtime evaluation. Because the code uses C++17 and the shared-library interface, it can be incorporated into a larger classical analysis program without depending on ROOT for the narrow mathematical kernel demonstrated here.

4. Numerical verification

Mathematically equivalent formulations

The independent programmatic reference uses the same formulas expressed in native C++ (`std::cosh`, `std::cos`, `std::sinh`, `std::sin`, and related functions). A second high-precision check computes reference values with `mpmath` using **110 decimal digits**. Fourteen near-collinear edge cases with `deta = dphi` from `10^-2` to `10^-15` are tested. The stable expression agrees with the high-precision reference near the accuracy expected of binary64 arithmetic in these selected cases, while the direct expression loses precision for sufficiently small angles.

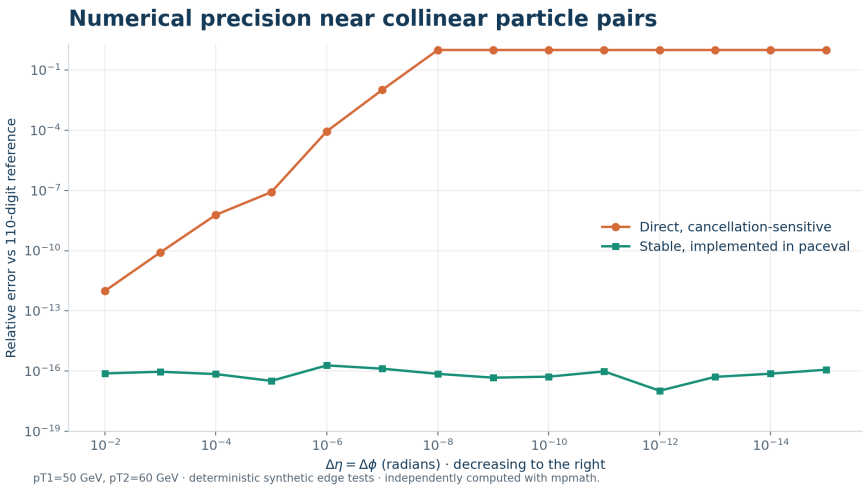


Figure 3. Numerical formulation study against an independent 110-digit reference. This is a precision comparison between mathematical expressions, not an execution-time comparison.

In the deterministic 120,000-event synthetic study, **12,317** cases showed substantial cancellation in the direct formula relative to a long-double reference (defined as more than `10^-6` relative error while the stable form remained below that threshold). In **5,144** cases the direct result was nonpositive although the stable-reference value was positive. At the intentionally artificial `m2 > 10^-10 GeV²` threshold, **eight** selection decisions differed. These counts characterize the supplied synthetic cases, not the prevalence of such events in an experimental detector.

Sampled TINC intervals

The actual paceval TINC calls returned a numerical value, lower bound and upper bound for each selected input. The test evaluates 12 angular points for each mass formula (24 interval results in total), using `pt1 = 50 GeV`, `pt2 = 60 GeV`, and equal `deta` and `dphi` from `10^-2` to `10^-13`. All **24/24** sampled intervals contained the independently computed 110-digit reference evaluated at the represented binary64 input values.

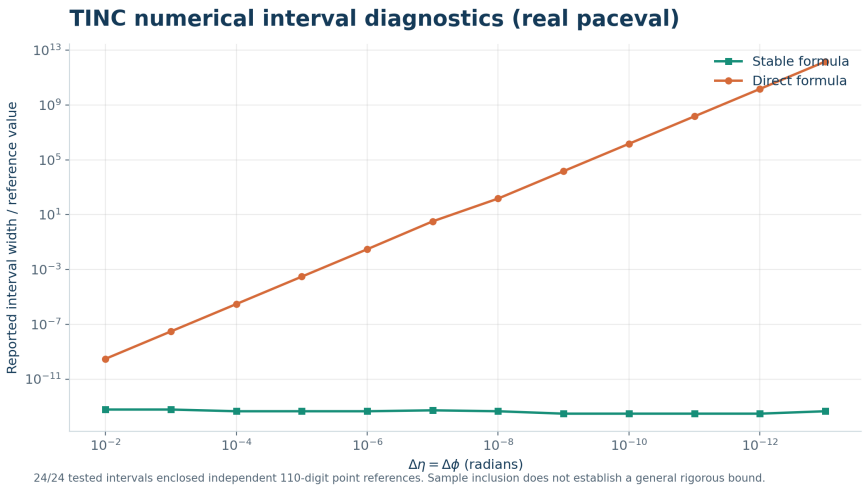


Figure 4. Original paceval-reported interval width as a numerical diagnostic; sample values are retained in examples/tinc_samples.csv.

The inclusion result is empirical for the selected points. **It does not establish a universal certified enclosure guarantee for TINC**, nor does it replace a dedicated rigorous interval-arithmetic proof. Numerical rounding error is distinct from experimental, statistical, systematic and physics-model uncertainties.

5. Integration and software validation

The demonstrator tests direct, stable, weight, fused, multigraph and systematic-variation results against its native C++ reference. The C++ code enforces a relative tolerance of **2×10^-8** and an absolute tolerance of **10^-28** for these comparisons. The recorded original-library runs produced **no mismatches outside those tolerances**. The program’s self-tests exercise algebraic consistency, near-collinear inputs, exact collinearity and 2,500 seeded synthetic samples.

Both four-column and eight-column CSV fixtures are provided: examples/example_4col.csv and examples/example_events.csv. This enables a future workflow to pass preselected reconstructed kinematic variables to the mathematical layer. No direct connection to CMS or ROOT is asserted by this demonstration.

Build and integration testing used **Debian GNU/Linux 13, x86-64, a KVM-virtualized AMD EPYC 9V74 environment, GCC 14.2 (C++17)** and independent **Clang 17** compilation checks. CTest includes a native mathematical self-test and, when the vendor library is supplied, an integration run using the original shared

runtime. Additional address- and undefined-behavior sanitizer checks were performed with the instrumented application. These environment details identify the verification conditions, not a hardware-dependent scientific claim.

The exact original library used for validation had SHA-256:

```
acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12
```

6. Reproduce the verified implementation

The public GitHub repository intentionally excludes the proprietary paceval shared library. A compatible version must be supplied separately, referenced by its absolute path. From the package root on Linux:

```
export PACEVAL_LIB=/absolute/path/to/libpaceval_linux_sharedLIB.so
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME="$PACEVAL_LIB"
cmake --build build --parallel
ctest --test-dir build --output-on-failure
mkdir -p out
./build/collider --library "$PACEVAL_LIB" --require-paceval \
  --input-csv examples/example_events.csv --batch 16 \
  --csv out/example_validation.csv
python3 src/high_precision_reference.py \
  --tinc-csv out/tinc_samples.csv --strict-tinc \
  --out out/high_precision_audit.csv
```

The independent checker requires Python 3 and `mpmath` (`python3 -m pip install mpmath`). It also contains a built-in fourteen-point high-precision edge study. The C++ application itself uses C++17 plus standard system dynamic loading; the original paceval library is required for paceval execution. The original TINC samples and high-precision reference CSV are included so the scientific checks can be audited without regenerating the event fixture.

7. Scientific conclusion and possible follow-up

The defined feasibility use case was implemented and validated with paceval. The work demonstrates persistent text-expression graphs for collider-inspired kinematics, reusable parameterized event-weight functions, batch calls, joint graph evaluation, fused expressions, and TINC interval diagnostics. Independent native and high-precision references support the numerical findings for the tested inputs.

The next scientific extension would replace the synthetic event weight with a separately approved physics formula and evaluate the same computation using publicly released collider-event samples within an established framework such as ROOT RDataFrame. Such physics-oriented integration, formal interval certification and implementation on dedicated FPGA/ASIC hardware remain separate research tasks, not outcomes established by this demonstration.

References and included artifacts

1. paceval UG, documented API and expression-engine documentation.
2. ROOT Collaboration, ROOT data-analysis framework.
3. CERN, CERN Open Data Portal.
4. The repository README and source files C++ demonstrator, CMake project, and high-precision independent audit.
5. The retained original-library TINC observations and independent reference observations.

Scope statement: This is an independent synthetic-data research demonstrator by paceval UG. No CERN, CMS or other experimental-collaboration endorsement or existing partnership is implied.